Project - Low Level Design on Mutli Insurance Creator

Course Name: GEN AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	YASH SHRIVASTAVA	EN22CS3011111
2.	VISHWAS BHARGAV	EN22CS3011095
3.	YASHI HARNE	EN22CS3011116
4.	DIXIKA VERMA	EN22ME304027
5.	ANVESHA WANKHADE	EN22ME304014

Group Name: Group 07D10

Project Number: GAI-20

Industry Mentor Name:

University Mentor Name: Prof. Divya Kumawat

Academic Year:2025-2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. References

1. Introduction

The Multimodal Insurance Creator leverages generative AI to transform textual insurance concepts into both detailed narratives and high-quality visual representations. This document provides a structured overview of the system, its purpose, and intended usage.

1.1. Scope of the Document

This document outlines the objectives, design, and functionality of the Multimodal Insurance Creator. It covers:

- Integration of Large Language Models (LLMs) and image generation APIs to create multimodal content.
- System architecture, workflows, and data handling for generating and managing outputs.
- Guidelines for using the system to produce coherent text and visuals for professional and educational applications.

Exclusions: Detailed internal implementations of third-party AI models are outside the scope; focus is on system-level orchestration and application use.

1.2. Intended Audience

This document is intended for:

- Insurance professionals seeking clear, visual representations of complex concepts.
- AI developers and data scientists implementing generative AI workflows.
- Educators and trainers designing instructional content.
- Project managers and stakeholders overseeing AI-based innovation initiatives.

It ensures that all readers understand the system's capabilities, relevance, and potential applications.

1.3. System Overview

The Multimodal Insurance Creator converts textual prompts into multi-dimensional outputs that combine narrative and visual information:

- Input Layer: Accepts textual prompts describing insurance concepts or

scenarios.

- Processing Layer:
  - ➤ Text-to-Text: Generates structured, context-aware narratives.
  - ➤ Text-to-Image: Produces visual representations such as risk infographics or conceptual diagrams.
- Output Layer: Delivers integrated multimodal content suitable for presentations, reports, and educational materials.
- Storage & Retrieval: Optional vector database enables indexing and efficient retrieval of generated outputs.

This system demonstrates the synergy of text and image generation models to provide comprehensive, professional-grade insights into insurance topics.

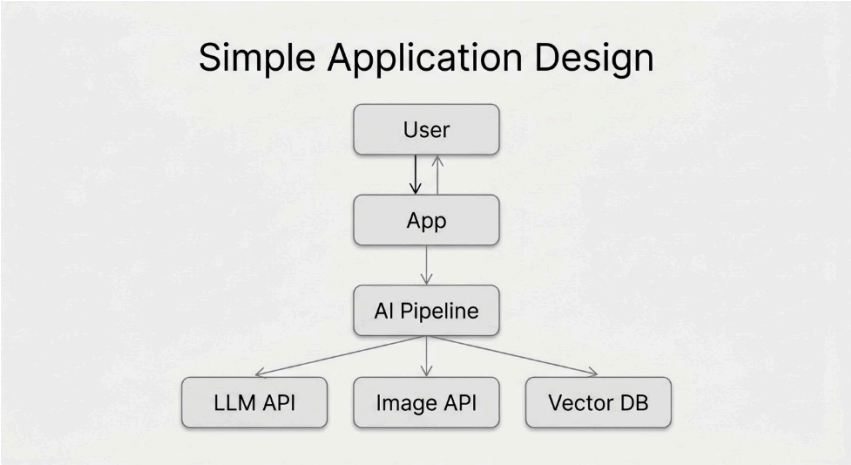
2. System Design.

2.1 Application Design

This diagram represents the architecture of a multimodal insurance application that generates both textual and visual outputs.

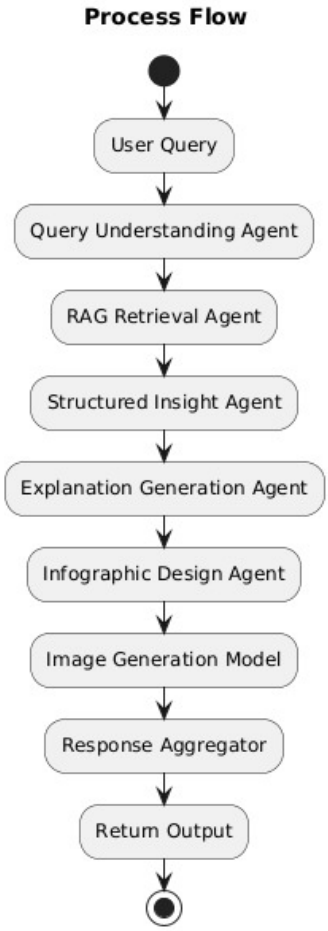
- User
  The user provides input such as an insurance query, policy request, or risk description.
- API Layer
  The API acts as an interface between the user and the system. It handles incoming requests and sends them to the AI pipeline, then returns the generated response back to the user.
- AI Pipeline
  This is the core component that processes the user input. It analyzes the request and routes it to appropriate AI services for generating results.
- LLM API (Large Language Model)
  Used to generate insurance-related textual content such as policy explanations, claim details, and risk analysis.
- Image API
  Generates visual representations like risk diagrams, insurance infographics, or claim illustrations based on user input.
- Vector Database
  Stores insurance-related knowledge in the form of embeddings. It

helps retrieve relevant information to improve response accuracy and context understanding.



2.2Process Flow

- Title: **Process Flow**
- Starts with a **start node (black circle)**
- **User Query** (user inputs request)
- **Query Understanding Agent** (interprets the query)
- **RAG Retrieval Agent** (fetches relevant information)
- **Structured Insight Agent** (analyzes structured data)
- **Explanation Generation Agent** (creates explanation)
- **Infographic Design Agent** (designs visual output)
- **Image Generation Model** (generates visuals/images)
- **Response Aggregator** (combines all outputs)
- **Return Output** (final response to user)
- Ends with an **end node (circle)**



2.3 Information Flow.

- Title: **Multimodal Insurance AI System Information Flow**
- Subtitle: Professional, modern, clean process diagram
- Starts with **User Query** (user asks a question)
-
- Next step: **Understand Intent** (AI interprets the request)

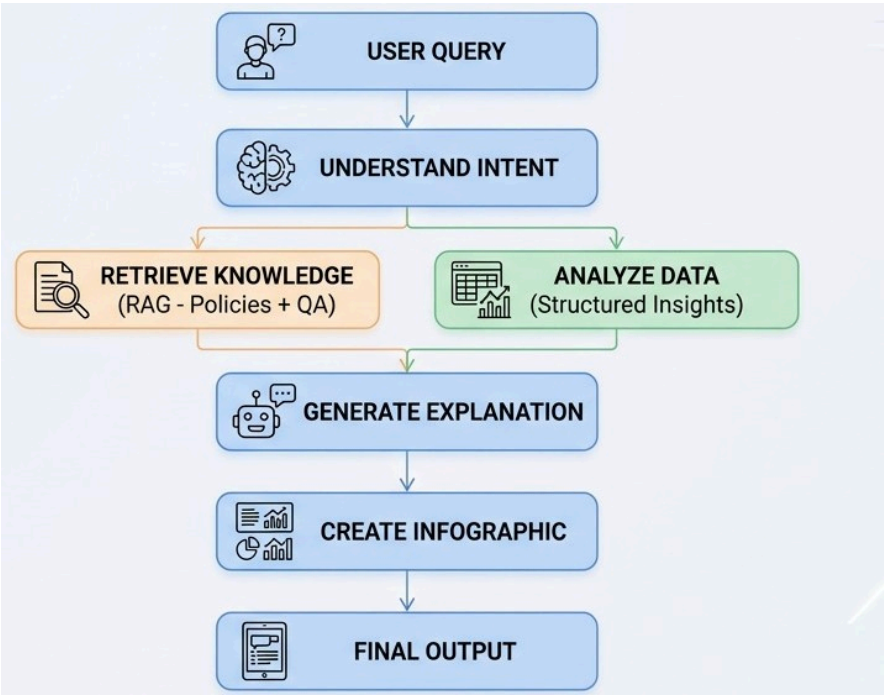
- Process splits i

  - **Retrieve Knowledge (RAG - Policies + QA)** (fetches relevant insurance info)
  - **Analyze Data (Structured Insights)** (processes structured data)

- Both paths merge into **Generate Explanation** (AI creates a response)

- Then: **Create Infographic** (visual representation of insights)

- Final step: **Final Output** (delivered to the user)

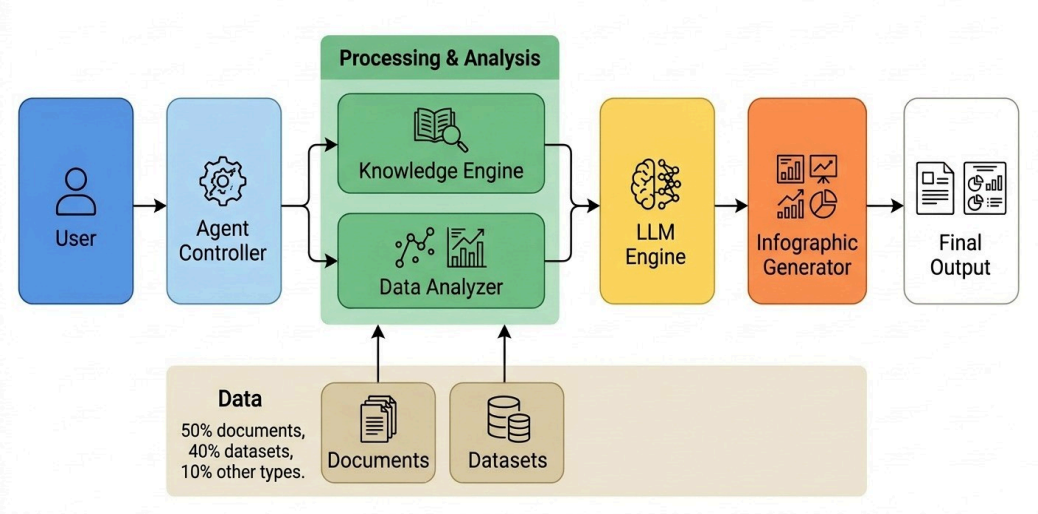


2.4 Components Design

- User Input: User provides query or request related to insurance.
- Agent Controller: Acts as a manager that receives the request and sends it to processing modules.
- Processing & Analysis Layer:
  - Knowledge Engine: Retrieves relevant insurance information.
  - Data Analyzer: Analyzes datasets and documents for insights.

- DataSources:
  - Documents (≈50%)
  - Datasets (≈40%)
  - Other data types (≈10%)
- LLM Engine (Large Language Model):
  Processes analyzed data and generates meaningful text output.
- Infographic Generator:
  Converts generated text into visual formats like charts, diagrams, or infographics.
- Final Output:

  Provides both text explanation + visual representation to the user.



2.5 Key Design Considerations

- Scalability:

Modular architecture allows horizontal scaling of AI agents and backend API for multiple concurrent users.
Vector DB and LLM API calls can be distributed to handle high traffic.

- Performance:

Efficient use of caching for repeated queries to reduce latency.
Asynchronous API calls to LLM and Image Generation services to improve responsiveness.

- Reliability:

Retry mechanisms for external API calls.

Graceful handling of partial failures (e.g., text generated but image API fails).

- Security:

Secure storage of API keys.
Input validation to prevent malicious prompts.
HTTPS for all client-server communications.

- Cost Optimization:

Use of token-efficient queries to LLM API.
Optional image resolution or batching to reduce API costs.

2.6 API Catalogue:

1. Internal API: Generate Multimodal Output

Endpoint: POST /generate

Description: Accepts a user query and returns a comprehensive multimodal response, including both textual
explanation and a visual infographic.

➤ Request Example:

JSON

```
{
"query": "Explain flood risk for coastal properties"
}
```

➤ Response Example:

JSON

```
{
  "narrative": "Flood risk includes storm surges, heavy rainfall, and rising sea levels. Proper insurance coverage is recommended to mitigate financial losses.",
  "image_url": "https://your-storage.com/generated-infographic.png"
}
```

2. Internal API: Health Check

Endpoint: GET /health

Description: Returns the status of the backend service to ensure the system is operational.

- Response Example:

JSON

```
{
"status": "OK"

}
```

3. API USED

API Name	Purpose	Description
LLM API	Text Generation	Used by the Explanation Generation Agent to produce detailed textual narratives.
Image Generation API	Infographic/Image Generation	Used by the Image Generation Model to create visual representations based on text prompts.
Vector Database API	Context Retrieval	Used by the RAG Retrieval Agent to fetch relevant contextual data using embeddings.

3.References

1. OpenAI. GPT-4 Technical Report. 2023. https://arxiv.org/abs/2303.08774

– Details the architecture and capabilities of large language models.

2. Ramesh, A., et al. Zero-Shot Text-to-Image Generation with DALL·E. NeurIPS, 2021. https://arxiv.org/abs/2102.12092

– Discusses text-to-image generative modeling techniques.

3. Brown, T., et al. Language Models are Few-Shot Learners. NeurIPS,

4. ..... 2020. https://arxiv.org/al

– Foundational paper on LLM capabilities.

5. Radford, A., et al. Learning Transferable Visual Models From Natural Language Supervision (CLIP). ICML, 2021. https://arxiv.org/abs/2103.00020

– Explains alignment of text and image embeddings.

6. Karras, T., et al. A Style-Based Generator Architecture for Generative Adversarial Networks. CVPR, 2019. https://arxiv.org/abs/1812.04948

– Key reference for high-quality image generation techniques.